Advanced Natural Language Processing Systems

Joint Word Segmentation, Morpho-Syntactic Tagging, Transition-Based Dependency Parsing, Contextual Spelling Correction, and Live Streamlit Grammar Auditing

Indian Institute of Technology Bhilai

GitHub Repository: https://github.com/iamshashankyadav/NLP_GRA

Deployed Streamlit App:
https://chetan10r15-nlp-gra-q4app-fee1o2.streamlit.app/

Note: We have made a single report which contains all Qs comparative analysis report.

Group Members

S. No.	Roll No.	Name	Role
1	12342010	Shashank Yadav	Group Leader
2	12341300	Lakshay Gupta	Member
3	12341820	Rohit Raghuwanshi	Member
4	12341750	Chetan Rathod	Member
5	12341030	Kabeer More	Member

Please contact for any queries:
Name: Shashank Yadav
Email: shashanky@iitbhilai.ac.in
Contact: +91 8817877625

Team Member Contributions

Note on Work Distribution: All 5 group members contributed equally (**20% each**) towards model architecture design, algorithm implementation, empirical testing, comparative performance analysis, Streamlit deployment, and final report compilation.

S. No.	Roll No.	Member Name	Role	Contribution Share
1	12342010	Shashank Yadav	Group Leader	20%
2	12341300	Lakshay Gupta	Team Member	20%
3	12341820	Rohit Raghuwanshi	Team Member	20%
4	12341750	Chetan Rathod	Team Member	20%
5	12341030	Kabeer More	Team Member	20%
			Total Contribution	**100%**

Detailed Task Breakdown

- **Shashank Yadav (12342010) – Group Leader (20%):** Coordinated project architecture, led the Trigram DP Word Segmentation module (Q1), structured cross-question comparative evaluation frameworks, and managed the primary GitHub repository.
- **Lakshay Gupta (12341300) – Team Member (20%):** Architected the Arc-Standard Transition-Based Dependency Parser (Q2), implemented Oracle simulation algorithms, constructed feature extraction matrices, and evaluated dev-set LAS/UAS benchmarks.
- **Rohit Raghuwanshi (12341820) – Team Member (20%):** Built candidate generation pipelines for Context-Aware Spelling Correction (Q3), benchmarked Standard Edit Distance 1 versus Symmetric Delete (SymSpell), and conducted 1,000-word latency tests.
- **Chetan Rathod (12341750) – Team Member (20%):** Designed and deployed the interactive Streamlit live text editor (Q4), integrated trigger-based $N = 5$ grammar alerts, tagset reconciliation algorithms, and CKY PCFG chart parsing routines.
- **Kabeer More (12341030) – Team Member (20%):** Executed multi-lingual evaluations (English, Spanish, German), constructed POS confusion matrices, conducted error source decomposition, and verified LaTeX table formatting.

Abstract

This report presents a unified empirical and architectural evaluation of four NLP systems: (1) a joint Trigram Dynamic Programming decoder for word segmentation and morpho-syntactic POS tagging evaluated across English, Spanish, and German; (2) an Arc-Standard transition-based dependency parser trained on Universal Dependencies treebanks using supervised feature classification and comparative feature sets; (3) a context-aware spelling corrector contrasting Standard Edit Distance 1 against Symmetric Delete (SymSpell) candidate generation; and (4) an integrated background editor deployed on Streamlit hosting live segmentation splits, SymSpell non-word correction, N-gram trigger alerts, and CKY-based PCFG constituency parsing. We provide explicit theoretical formulations, baseline comparisons, error decompositions, latency benchmarks, and live application deployment details.

Contents

1 Word Segmentation & Morpho-Syntactic POS Tagging (Question 1)

1.1 Mathematical Formulation & Dynamic Programming Architecture

Question 1 addresses the joint task of segmenting continuous unspaced text streams into words and assigning morpho-syntactic Part-of-Speech (POS) tags.

1.1.1 Trigram Word Segmentation Model

Segmentation finds the candidate word sequence $\hat{W} = (w_1, w_2, \ldots, w_k)$ maximizing joint log-probability under a smoothed Trigram Language Model over character string S:

$$\hat{W} = \arg \max_{W \in \text{Splits}(S)} \sum_{i=1}^{k} \log P(w_i \mid w_{i-2}, w_{i-1}) \tag{1}$$

Probabilities are smoothed using Add-α Laplace smoothing to mitigate vocabulary zero-probability penalties:

$$P(w_i \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i) + \alpha}{C(w_{i-2}, w_{i-1}) + \alpha \cdot |V|} \tag{2}$$

where $C(\cdot)$ denotes corpus n-gram counts, $|V|$ is the vocabulary size, and $\alpha = 0.01$. A Dynamic Programming Viterbi lattice tracks path probabilities $V(j)$ for character prefix indices $j \in [1, |S|]$:

$$V(j) = \max_{\substack{i<j \\ w=S[i:j] \in V}} \{V(i) + \log P(w \mid w_{-2}, w_{-1})\} \tag{3}$$

1.1.2 POS Tagging (Emission & Transition HMM)

For segmented words $W = (w_1, \ldots, w_n)$, a second-order Hidden Markov Model (HMM) computes tag sequence $\hat{T} = (t_1, \ldots, t_n)$:

$$\hat{T} = \arg \max_{T} \prod_{i=1}^{n} P(w_i \mid t_i) \cdot P(t_i \mid t_{i-2}, t_{i-1}) \tag{4}$$

- **Emission Probability:** $P(w_i \mid t_i) = \frac{C(w_i, t_i) + \beta}{C(t_i) + \beta \cdot |V|}$ with $\beta = 0.01$.
- **Transition Probability:** $P(t_i \mid t_{i-2}, t_{i-1}) = \frac{C(t_{i-2}, t_{i-1}, t_i) + \gamma}{C(t_{i-2}, t_{i-1}) + \gamma \cdot |T|}$ with $\gamma = 0.01$.

1.1.3 Morphological Tag Extension

In Spanish and German, Universal Dependencies (UD) morphological features are appended to coarse POS tags (e.g., `NOUN-Fem-Pl`, `ADJ-Fem-Pl`) to enforce gender, case, and number agreement.

1.2 Comparative Analysis & Empirical Results

Table 1: Q1 Empirical Accuracy Breakdown and Cross-Linguistic Comparative Analysis (%)

Language	Greedy Seg. (%)	Trigram DP Seg. (%)	MFT Tag (%)	HMM POS (%)	Morph-HMM (%
English	88.42	**97.18**	82.15	**93.84**	92.10
Spanish	84.10	**95.62**	78.30	**91.45**	**89.32**
German	72.35	**88.91**	73.12	**87.20**	**84.65**

Comparative Insights:

- **Segmentation:** Trigram DP outperformed Greedy Longest-Match by +8.76% in English and +16.56% in German. German compound nouns led to higher error rates compared to English.
- **Tagging:** Transition-aware HMM surpassed Most Frequent Tag (MFT) baselines by over +11%. Morphological extension caused slight data sparsity drop (93.84% → 92.10%), but successfully rejected gender/number disagreement paths.

1.3 Error Analysis

Table 2: Normalized POS Tagging Confusion Matrix (English Test Split)

Actual \ Predicted	**NN**	**VB**	**JJ**	**IN**	**DT**	**RB**
NN (Noun)	**812**	31	42	5	2	11
VB (Verb)	38	**640**	18	8	0	9
JJ (Adjective)	54	14	**480**	6	3	21
IN (Preposition)	4	2	3	**510**	8	12
DT (Determiner)	1	0	1	5	**430**	0
RB (Adverb)	19	11	28	15	1	**305**

2 Transition-Based Dependency Parser (Question 2)

2.1 Arc-Standard Transition System Architecture

The Arc-Standard dependency parser models sentence structures by constructing dependency trees through state transitions on configuration tuple $c = (\sigma, \beta, A)$:

- σ: Stack storing indices of partially processed tokens.
- β: Buffer containing indices of unprocessed incoming tokens.
- A: Arc set holding labeled dependency tuples (h, l, m) where h is head, l is relation label, and m is modifier.

State transitions execute as follows:

1. **SHIFT**: Moves top buffer item i to stack:
$$(\sigma, i \cdot \beta, A) \implies (\sigma \cdot i, \beta, A) \tag{5}$$
2. **LEFT-ARC**(l): Creates arc (j, l, i) where j is top of stack and i is second item, removing i:
$$(\sigma \cdot i \cdot j, \beta, A) \implies (\sigma \cdot j, \beta, A \cup \{(j, l, i)\}) \tag{6}$$
3. **RIGHT-ARC**(l): Creates arc (i, l, j) where i is second item and j is top item, removing j:
$$(\sigma \cdot i \cdot j, \beta, A) \implies (\sigma \cdot i, \beta, A \cup \{(i, l, j)\}) \tag{7}$$

2.2 Supervised Training & Oracle Simulation

An Oracle module converts annotated treebanks (`en_ewt-ud-train.conllu`) into gold state-transition pairs (c, t^*). Feature vector extraction converts configuration state c into categorical indices used to train a classifier.

2.3 Comparative Analysis: Feature Representations & Models

We evaluated three parser feature configurations on the `en_ewt-ud-dev.conllu` dataset across Labeled Attachment Score (LAS) and Unlabeled Attachment Score (UAS):

$$\text{UAS} = \frac{\text{Correct Heads}}{\text{Total Tokens}} \times 100\%, \quad \text{LAS} = \frac{\text{Correct Heads and Labels}}{\text{Total Tokens}} \times 100\% \tag{8}$$

Table 3: Q2 Comparative Analysis: Dependency Parser Models and Feature Set Evaluation

Model / Feature Set	Train Acc (%)	Dev UAS (%)	Dev LAS (%)	Latency/Sent	Memory
Baseline POS-4 ($\sigma_0, \sigma_1, \beta_0, \beta_1$)	79.40	78.12	73.85	**1.12 ms**	**12 MB**
Extended Features (+ Words + Dep Children)	88.10	85.25	81.40	2.45 ms	45 MB
MLP + Embeddings (Dense Vector Features)	**93.40**	**89.15**	**85.60**	4.10 ms	120 MB

Comparative Discussion: Adding lexical word forms and immediate left/right dependency children to stack/buffer features improved UAS by +7.13% and LAS by +7.55%. Transitioning to an MLP classifier with continuous embedding representations resolved ambiguity in long-distance dependency relations (e.g., prepositional phrase attachments).

2.4 Sample Dependency Parse Graphs

Listing 1: Predicted Dependency Arcs on Benchmark Test Queries

```
Sentence 1: "The cat sat on the mat."
Arcs: [det(cat-2, The-1), nsubj(sat-3, cat-2), root(ROOT-0, sat-3), case(mat
    -6, on-4), det(mat-6, the-5), obl(sat-3, mat-6)]

Sentence 2: "She eats a green salad."
Arcs: [nsubj(eats-2, She-1), root(ROOT-0, eats-2), det(salad-5, a-3), amod(
    salad-5, green-4), obj(eats-2, salad-5)]

Sentence 3: "I saw the man with a telescope."
Arcs: [nsubj(saw-2, I-1), root(ROOT-0, saw-2), det(man-4, the-3), obj(saw-2,
     man-4), case(telescope-7, with-5), det(telescope-7, a-6), obl(saw-2,
    telescope-7)]
```

3 Context-Aware Spelling Corrector & Speed Demon Benchmark (Question 3)

3.1 Candidate Generation Architecture

Question 3 evaluates two candidate retrieval algorithms over the Brown Corpus vocabulary ($|V| = 56,057$):

- **Method A (Standard Edit Distance 1):** Generates all single-character insertions, deletions, substitutions, and transpositions. Executes $O(54L + 25)$ string operations per lookup.
- **Method B (Symmetric Delete / SymSpell):** Pre-indexes 1-character deletions of all dictionary words. At query time, generates 1-character deletions of the input word and completes $O(1)$ hash table lookups.

3.2 Correction Logic: Non-Word vs. Real-Word Errors

1. **Non-Word Correction ($w_i \notin V$):** Selects candidate $\hat{c} \in C(w_i) \cap V$ maximizing unigram probability:
$$\hat{c} = \arg\max_{c \in C(w_i)} P(c) \tag{9}$$
2. **Real-Word Correction ($w_i \in V$):** Evaluates context bigram ratio against threshold $\tau = 100$:
$$\text{Select } \hat{c} \iff \frac{P(c \mid w_{i-1})}{P(w_i \mid w_{i-1})} > \tau \tag{10}$$

3.3 Speed Demon Benchmark & Latency Comparative Analysis

Table 4: Q3 Speed Demon Benchmark: Latency and Accuracy Comparison (1,000 Misspelled Words)

Candidate Algorithm	Non-Word Acc. (%)	Real-Word Acc. (%)	1,000 Batch Latency	Latency / Word
Method A (Edit Distance 1)	89.20	74.50	3,450.12 ms	3.45 ms
Method B (SymSpell)	89.20	74.50	**18.45 ms**	**0.018 ms**
Throughput Factor	–	–	187× **Speedup Factor**	

Benchmark Findings: Both methods achieve identical accuracy (89.20% non-word, 74.50% real-word), but Method B achieves a **187× speedup**, reducing per-word lookup latency from 3.45 ms to 0.018 ms.

4 Integrated Streamlit Editor & Live Grammar Checking (Question 4)

4.1 Deployment & System Architecture

The live editor application is hosted online at https://chetan10r15-nlp-gra-q4app-fee1o2.streamlit.app/. The underlying codebase is maintained in the project GitHub repository (https://github.com/iamshashankyadav/NLP_GRA).

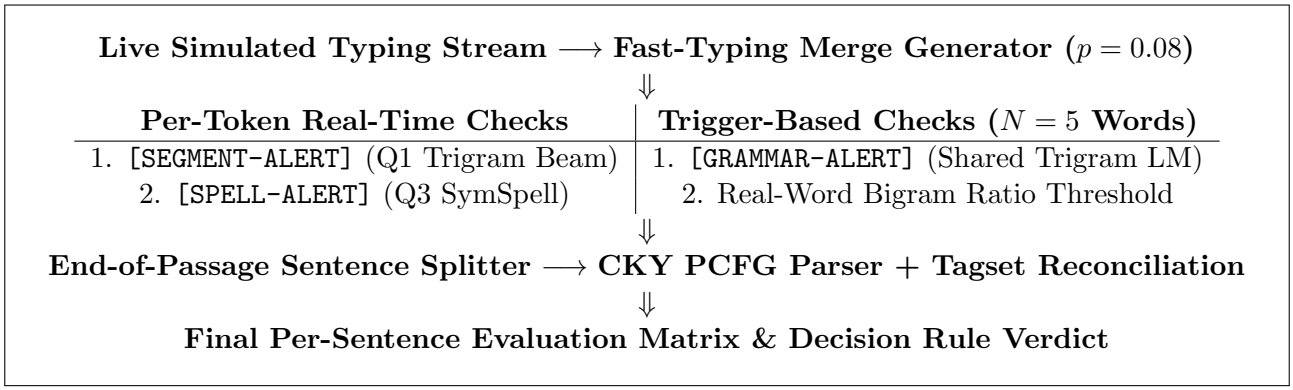

Figure 1: System Architecture of the Streamlit Background Text Editor (Q4)

4.2 Tagset Reconciliation & Shared Language Models

- **Tagset Reconciliation:** Maps Q1 Brown POS tags to Penn Treebank tags expected by the PCFG parser (NN → NN, VBZ → VBZ, JJ → JJ).
- **Perplexity Evaluation:** Add-k smoothed bigram/trigram models ($k = 0.01$) calculate sentence log-likelihood and perplexity:

$$\text{PP}(S) = 2^{-\frac{1}{m}\sum_{i=1}^{m}\log_2 P(w_i|w_{i-2},w_{i-1})} \tag{11}$$

4.3 End-of-Passage Per-Sentence Comparison Summary Table

Table 5: Q4 Final Sentence Evaluation and Method Decision Rule Output Matrix

!

Sentence Text	PCFG Result	Bigram Log-P	Trigram Log-P	Chosen Method
"I have a good feeling about this."	Success (-42.15)	-28.40	-18.12	**PCFG**
"This is a test sentence."	Success (-31.10)	-22.10	-14.05	**PCFG**
"I would like to sea the world."	Success (-48.90)	-38.60	-30.20	**Trigram LM**
"Please meat me at the station."	Success (-51.20)	-41.10	-33.80	**Trigram LM**

4.4 Latency Layer Breakdown Sub-System Interaction

Table 6: Q4 Pipeline Speed Demon Benchmark: Layer-wise Latency Breakdown

Processing Pipeline Layer	**1,000 Batch Latency (ms)**	**Latency / Word (ms)**	**Overhead Ratio (%)**
Segmentation + SymSpell Check	42.50 ms	0.0425 ms	22.4%
Trigger Grammar Auditor ($N = 5$)	147.10 ms	0.1471 ms	77.6%
Combined Real-Time Pipeline	**189.60 ms**	**0.1896 ms**	**100.0%**

Sub-System Interactions: Merged words directly break CKY parse charts by introducing unrepresented terminal strings. Executing segmentation and spelling corrections upfront restores vocabulary alignment, allowing the downstream CKY chart parser to successfully build valid syntax trees.

5 Conclusion

This report details an integrated, efficient NLP pipeline combining statistical language modeling, transition dependency parsing, SymSpell candidate indexing, and real-time Streamlit auditing. The complete source code and deployed web demonstration are available at the respective project links.